ASSIGNMENT-3.5

A.SANDEEP

2403A52L03

B-50

Task 1

Zero-Shot Prompting (Leap Year Check)

Write a zero-shot prompt to generate a Python function that checks

whether a given year is a leap year.

Week2 -

Task:

• Record the AI-generated code.

• Test with years like 1900, 2000, 2024.

• Identify logical flaws or missing conditions.

Prompt:

Generate a Python function to check whether a given year is a leap year and return True or False.

Code:

```
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    return False
# Testing
years = [1900, 2000, 2024]
for y in years:
    print(y, ":", is_leap_year(y))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/AI CODING/ai
1900 : False
2000 : True
2024 : True
PS C:\Users\prash>
```

Code Explanation:

A leap year is divisible by 4 but not by 100 unless divisible by 400.

Example:

1900 → Not leap

2000 → Leap

2024 → Leap

Task 2

One-Shot Prompting (GCD of Two Numbers)

Write a one-shot prompt with one example to generate a Python

function that finds the Greatest Common Divisor (GCD) of two numbers.

Example:

Input: 12, 18 → Output: 6

Task:

• Compare with a zero-shot solution.

• Analyze algorithm efficiency.

Prompt:

Generate a Python function to find GCD of two numbers. Example: Input 12, 18 → Output 6.

Code:

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
```

```
    return a
print(gcd(12, 18))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/AI CODING/ai
6
PS C:\Users\prash>
```

Code Explanation:

Uses Euclidean Algorithm → very efficient → O(log n).

Task 3

Few-Shot Prompting (LCM Calculation)

Write a few-shot prompt with multiple examples to generate a Python

function that computes the Least Common Multiple (LCM).

Examples:

• Input: 4, 6 → Output: 12

• Input: 5, 10 → Output: 10

• Input: 7, 3 → Output: 21

Task:

• Examine how examples guide formula selection.

• Test edge cases.

Prompt:

Generate Python function to compute LCM using user input and given examples.

Code:

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
def lcm(a, b):
```

```
    return (a * b) // gcd(a, b)
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("LCM:", lcm(a, b))
```

Output:

```
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/pras
Enter first number: 7
Enter second number: 42
LCM: 42
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/pras
Enter first number: 4
Enter second number: 6
LCM: 12
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/pras
Enter first number: 5
Enter second number: 10
LCM: 10
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/pras
Enter first number: 7
Enter second number: 3
LCM: 21
```

Code Explanation:

This program calculates the Least Common Multiple (LCM) of two numbers using user input. It uses the mathematical relationship between LCM and GCD, where LCM = (a × b) / GCD(a, b). The program first calculates GCD using the Euclidean algorithm and then uses it to calculate LCM. Few-shot prompting is used because multiple examples help AI understand the correct formula and logic. This program is useful in scheduling problems, number theory, and mathematical computations..

Task 4

Zero-Shot Prompting (Binary to Decimal Conversion)

Write a zero-shot prompt to generate a Python function that converts a

binary number to decimal.

Task:

• Test with valid and invalid binary inputs.

• Identify missing validation logic.

Prompt:

Generate Python function to convert binary to decimal using user input.

Code:

```
# Task 4: AI Tool Comparison with User Input


tools = {
    "copilot": {
        "Name": "GitHub Copilot",
        "Usability": "Very Easy (IDE Integrated)",
        "Code Quality": "High"
    },
    "gemini": {
        "Name": "Gemini",
        "Usability": "Easy (Chat Based)",
        "Code Quality": "Good"
    },
    "cursor": {
        "Name": "Cursor AI",
        "Usability": "Moderate",
        "Code Quality": "Good"
    }
}


choice = input("Enter tool (copilot/gemini/cursor): ").lower()


if choice in tools:
    print("\nTool Details:")
    for key, value in tools[choice].items():
        print(f"{key}: {value}")
else:
```

```
    print("Invalid Tool Choice")
```

Output:

```
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/
Enter binary number: 4
Decimal: Invalid Binary Number
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/
Enter binary number: 0010010110
Decimal: 150
PS C:\Users\prash>
```

Code explanation:

Gemini is strong in explanations and concept learning. GitHub Copilot is best for real-time coding assistance inside IDEs and provides high-quality code suggestions. Cursor AI is good for prompt-based complete code generation and editing. Each tool is useful depending on the user's need, such as learning, coding speed, or full project generation.

Task 5

One-Shot Prompting (Decimal to Binary Conversion)

Write a one-shot prompt with an example to generate a Python function

that converts a decimal number to binary.

Example:

Input: 10 → Output: 1010

Task:

• Compare clarity with zero-shot output.

• Analyze handling of zero and negative numbers.

Prompt:

Generate Python function to convert decimal to binary using user input. Example: 10 → 1010.

Code:

```
def decimal_to_binary(n):
    return bin(n).replace("0b", "")
num = int(input("Enter decimal number: "))
print("Binary:", decimal_to_binary(num))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Do
Enter decimal number: 10
Binary: 1010
PS C:\Users\prash>
```

Code explanation:

This program converts a decimal number into binary using user input. It uses Python's built-in bin() function and removes the prefix "0b" to show only the binary digits. One-shot prompting helps AI generate correct conversion logic using one example (10 → 1010). This program is useful in computer science applications where binary representation is required, such as memory addressing and data storage.

Task 6

Few-Shot Prompting (Harshad Number Check)

Write a few-shot prompt to generate a Python function that checks

whether a number is a Harshad (Niven) number.

Examples:

• Input: 18 → Output: Harshad Number

• Input: 21 → Output: Harshad Number

• Input: 19 → Output: Not a Harshad Number

Task:

• Test boundary conditions.

• Evaluate robustness

Prompt:

Generate Python function to check Harshad number using user input and examples.

Code:

```
def is_harshad(n):
    digit_sum = sum(int(d) for d in str(n))
    if n % digit_sum == 0:
```

```
        return "Harshad Number"
    return "Not a Harshad Number"
num = int(input("Enter number: "))
print(is_harshad(num))
```

Output:

```
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/AI CODING/ai-coding-3.5.py"
Enter number: 18
Harshad Number
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/AI CODING/ai-coding-3.5.py"
Enter number: 21
Harshad Number
PS C:\Users\prash> & "C:/Program Files/Python313/python.exe" "c:/Users/prash/OneDrive/Documents/AI CODING/ai-coding-3.5.py"
Enter number: 19
Not a Harshad Number
PS C:\Users\prash>
```

Code explanation:

This program checks whether a number is a Harshad number using user input. A Harshad number is a number that is divisible by the sum of its digits. The program converts the number into digits, calculates their sum, and checks divisibility. Few-shot prompting helps AI understand the pattern using multiple examples. This program is useful in number theory and mathematical pattern analysis.